Linear Regression using the Normal Equation
Implementation, Performance and Scaling Analysis

Ollscoil Teicneolaíochta an Atlantaigh
Atlantic Technological University

What I implemented

- Implemented **linear regression from scratch** using the **normal equation** (no ML libraries).
- Used **NumPy** for matrix operations and numerical stability.

Data and model setup

- Dataset: real-world housing data (**1000 observations**, all numerical).
- Feature: **Square_Footage (X)**
- Target: **House_Price (y)**

$y = aX + b$

Main computation:

β = np.linalg.solve(X^TX, X^Ty)

Solves $(X^TX)\beta = X^Ty$

Model Pipeline:

X (features) + y (target)

↓

β = solution of $(X^TX)^{-1} X^Ty$

a, b extracted from β (slope, intercept)

↓

$\hat{y}$ (predictions) = $aX + b$

↓

RMSE (prediction error)

✓ Solving the linear system avoids explicit matrix inversion and improves numerical stability.

Model Results and *Interpretation*

Ollscoil Teicneolaíochta an Atlantaigh

Atlantic Technological University

Estimated parameters

a (slope) ≈ 200

→ Each additional square foot increases the predicted house price by about **200**.

b (intercept) ≈ 55,000

→ The intercept is a theoretical starting value when size is zero and is less important than the slope.

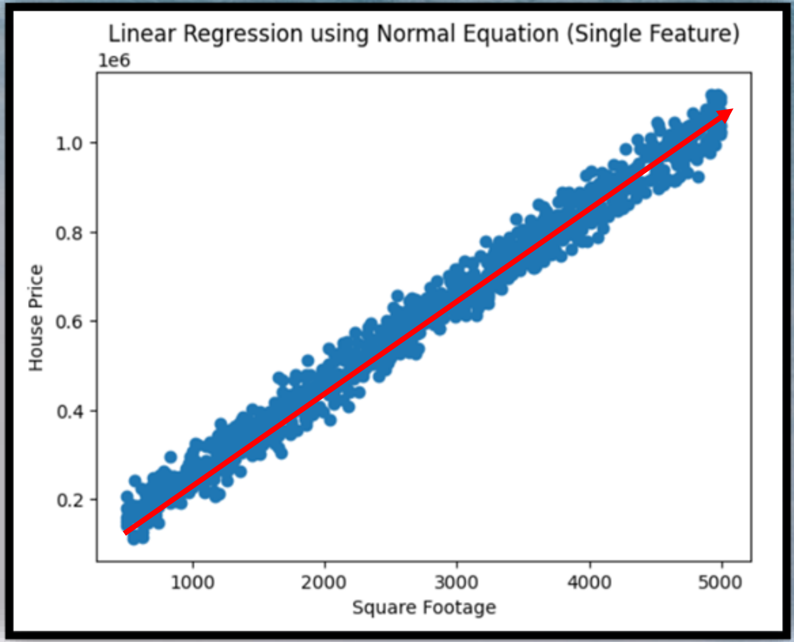
Model error (from data)

```
rmse = np.sqrt(np.mean((y - y_hat) ** 2))
```

RMSE ≈ 33,431
Mean house price ≈ 618,000
Relative error ≈ 5.4%

Train / test evaluation

Data split: **80% training / 20% test**
Model fitted on training data only
Test RMSE ≈ 32,900, consistent with overall error



Interpretation

- ✓ The regression line captures the main linear relationship between house size and price.
- ✓ The scatter around the line shows unexplained variation, caused by factors not included in the model.
- ✓ A single-feature linear model explains the overall trend well, but cannot explain all variability.

Computational Scaling and Limitations of the Normal Equation

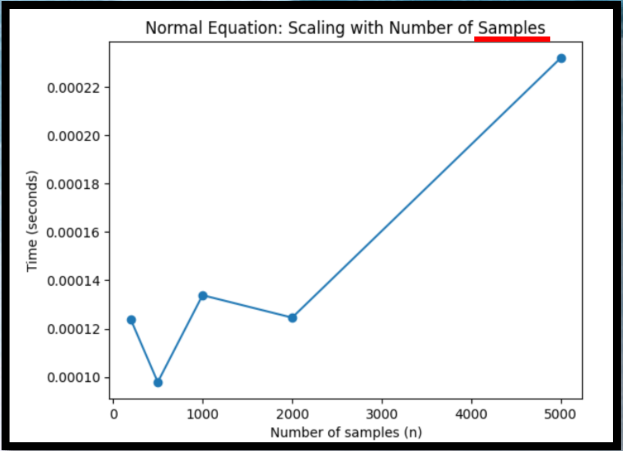
- Synthetic data was used to control the number of samples and features separately.

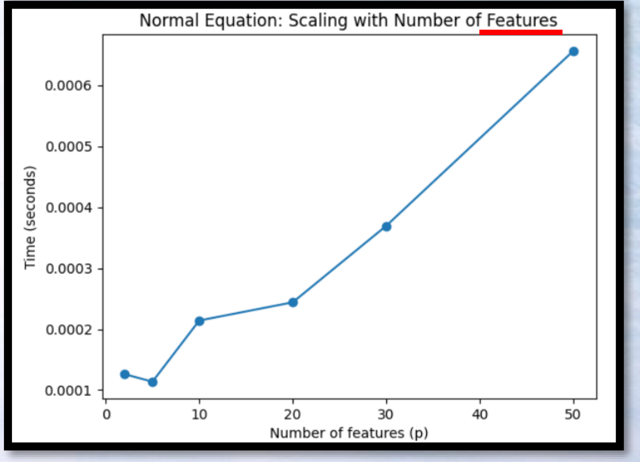
Sample size scaling(fixed p = 10): Runtime increased slowly (~0.00010s → ~0.00023s).

Feature size scaling (fixed n = 2000): Runtime increased rapidly (~0.00011s → ~0.00068s).

Main computation(solving the normal equation):

β = np.linalg.solve(X.T @ X, X.T @ y)





Key observations

- ✓ The normal equation solves a p × p linear system.
- ✓ As the number of features increases, this system becomes large and expensive.
- ✓ Computation time mainly depends on the number of features, not samples.

Conclusion

- ✓ The normal equation works well for small feature sizes.
- ✓ It does not scale well when the number of features increases.
- ✓ For large feature sets, other methods are more suitable

Note: Small variations are expected due to very small runtime measurements. The overall trend is the key result.